

Note-set demo, part 1 (Lecture 01) — a specification, its realizations, three kinds of verifier

Part 1 of a demonstration that runs across Lectures 01 and 02. In this part the repository holds one specification, `note-format-spec.md`, and nothing that realizes it. The class watches the specification audited and amended, then a realization verified and repaired, then a program written that performs the same verification. The concept of operations, the agent's operations on the set, and the invariant are part 2.

A caution that governs the whole script. Do not depend on the agent making one specific move. Agent behavior varies from run to run and from version to version. The *Expected* notes describe the usual shape, and every segment has an *If it goes differently* note. Rehearse the full run at least once before class; the rehearsal, not this script, is the record of what your setup does. Capture screenshots at every [fallback capture] marker during rehearsal, so that a live run that goes differently does not lose a teaching point.

What the starter contains, and what is seeded in it

`starter/` has three things: `note-format-spec.md` at version 0.1 (draft), the loader `CLAUDE.md`, and `process/` with three documents — `spec-audit.md` (AUD), `reporting.md` (RPT), `verification.md` (VER). No concept of operations; no notes; no `index.md`.

The specification contains three findings for the audit in Segment 2 to make. Do not reveal them before Segment 2.

- **Finding A — inconsistent within the document (AUD-2).** The scope sentence says “every markdown file in the note set MUST satisfy the rules below,” and R6 names `index.md`, a file with no front-matter that cannot satisfy R1.
- **Finding B — two clauses, one property, no stated relationship (AUD-2).** R2 requires a `title` field; R3 requires a level-1 heading; nothing says they agree.
- **Finding C — an execution gap (AUD-4).** R6 requires links to note files; nothing says what filename a note has. Writing one conformant note on paper means inventing a filename.

The date format is not a gap: R2 pins ISO-8601 from 0.1, so that in Segment 3 an original rule and an amended rule each catch a violation on the same note.

Before class — setup checklist

1. Create the demo repository from `starter/` and make the initial commit:

```
rm -rf ~/note-set-demo
cp -R <path-to-this-demo>/starter ~/note-set-demo
cd ~/note-set-demo
git init && git add -A
git commit -m "demo start: specification 0.1 and process"
```

2. Confirm `python3` runs (Segment 4 uses the standard library only).
3. Copy the 1.0.0 verifier from `starter-102/check_notes.py` to a folder outside the repository, as the Segment 4 fallback.
4. Open `sample-inputs/royce-note-filled-nonconformant.md` in a plain-text editor. A markdown-aware editor may repair the seeded heading problems on paste, which would remove Segment 3's findings.
5. Start Claude Code in `~/note-set-demo` in its default permission mode and confirm it loaded `CLAUDE.md` and the two always-on process files. Two terminals, large font: one for Claude Code, one for `cat`, `git`, `python3`.
6. Rehearse once; capture at every [fallback capture]; reset by repeating step 1.

Segment 1 — Tour (about 6 minutes)

Do (shell): `cat note-format-spec.md`.

Say: a normative specification: RFC 2119 keywords, numbered rules R1–R6, **Version: 0.1 (draft)**. Read R1–R6 aloud; it takes under a minute. The numbers exist so that a report — from a person, an agent, or a program — can cite a rule.

Do (shell): `cat CLAUDE.md`, then `ls process/` and `grep '^### ' process/*.md`.

Say: CLAUDE.md names one governing document and three process documents, and states two rules: read everything before acting; change nothing without approval. The process documents say how work proceeds with respect to the specification. Two are always loaded (RPT, how findings are reported; VER, how verification is done); one is run when asked (AUD, how a specification’s quality is assessed). Read the seven AUD headings aloud: these are the quality properties, as a procedure. Do not reveal the seeded findings.

[fallback capture] — the specification and the AUD headings on screen.

Segment 2 — The audit (about 14 minutes)

Do: switch Claude Code to plan mode (Shift+Tab cycles modes; the UI differs by version — say so). Plan mode is read-only.

Do: prompt:

Read `note-format-spec.md` and the process documents. Run the audit in `process/spec-audit.md` on the specification and report per RPT-1: number every finding, place it, quote the text, categorize it, and give a recommended resolution. Include the walkthrough that AUD-4 asks for: write one conformant note on paper, step by step, and say what you had to invent. State which of the three places in AUD-6 you searched. Then stop and wait for my rulings.

Expected: visible reads of the specification and the process files, then an RPT-1 list containing at least Findings A, B, and C, usually more.

[fallback capture] — the list.

Do: rule on the seeded findings with these sentences; do not improvise:

1. “`index.md` is not a note. Add a scope statement: R1–R5 govern the files in `notes/`; `index.md` is governed only by R6.”
2. “The H1 **MUST** be identical to the front-matter `title`. The field is authoritative: a repair adjusts the heading, never the field.”
3. “Add R7: a note’s filename is the slug of its title plus `.md` — lowercase the title, remove apostrophes, replace every remaining run of characters other than ASCII letters and digits with one hyphen, strip leading and trailing hyphens.”

For any further finding: rule in one sentence, or say “deferred” and let the RPT-1 status record it. (Deferred questions get a file of their own in part 2.)

Do: prompt:

Propose the amendments to `note-format-spec.md` per RPT-3 — old text, new text, rationale citing the finding, version bump, changelog entry — and wait for my approval.

Do: approve. Leave plan mode so the agent can edit the file, set the version to 1.0.0, add the changelog entry, and commit as `spec: 0.1 -> 1.0.0`.

Do (shell): `git diff HEAD~1 -- note-format-spec.md; git log --oneline`.

Say: the specification was wrong in three places before anything realized it. The audit found them; a person ruled; the changelog records the rulings and why. The specification moved; there was no realization to move yet.

If it goes differently: a missed finding gets one nudge — A: “R6 names `index.md`. Check it against R1.” B: “Can a conformant note’s H1 differ from its `title` field?” C: “Write one conformant note on paper. What filename does it get?” If plan mode misbehaves, run the same prompt in normal mode with “Do not create or modify any file until I approve” appended.

Segment 3 — The verification gate (about 14 minutes)

Do (shell and editor, agent idle):

```
mkdir notes
```

Paste the whole of `sample-inputs/royce-note-filled-nonconformant.md` into a new file `notes/royce-1970-waterfall-paper.md` — say that the filename follows the R7 you just approved — and write `index.md` by hand:

```
# Note Set Index
```

```
Study notes on software-engineering classics.
```

```
- [Royce 1970 Waterfall Paper](notes/royce-1970-waterfall-paper.md)
```

Say: “I wrote this note last night, in another editor, with nothing checking it.” For your eyes, the paste contains exactly four violations:

#	In the file	Rule	Repair
1	<code>created: Sept 11, 2026</code>	R2 (original, 0.1)	2026-09-11
2	<code>H1 # The Waterfall Paper</code> versus <code>title: Royce 1970 Waterfall Paper</code>	R3 (amended in Segment 2)	rewrite the H1; the field is authoritative
3	<code>a second H1, # My take</code>	R4	demote to <code>##</code>
4	<code>## What Royce actually said followed by ####</code>	R5	demote to <code>###</code>

Do: prompt:

I wrote `notes/royce-1970-waterfall-paper.md` myself and added its index entry by hand. Check the note and `index.md` for conformance to `note-format-spec.md` 1.0.0 and report per RPT-2. Do not change anything.

Expected: an RPT-2 report: verifier (agent, model, date); specification and version; clauses checked; four findings citing R2, R3, R4, R5 with the rule text quoted; a verdict; repairs proposed, none applied (RPT-4).

[fallback capture] — the report.

Say: the R3 finding exists only because of Segment 2; the 0.1 specification would have accepted this heading. The agent read the specification before judging (`CLAUDE.md` rule 1): the check is driven by the document, not by memory of it. The report’s first three lines — which verifier, which version, which clauses — are what makes the result usable later (VER-1).

Do: prompt:

Ruling: the specification is right and the note is wrong. Repair the note, keeping my wording except where conformance requires a change. Re-check, report per RPT-2, and commit as **repair: royce note** (R2, R3, R4, R5).

Expected: the four repairs; a passing re-check; a commit.

Say: in Segment 2 the specification moved and there was nothing to repair; here the realization moved and the specification held still. Both ended with conformance re-established. The passing re-check is the completion condition: the task was not “edit the file” but “make the check pass.”

If it goes differently: if the agent repairs without waiting, do not argue; `git diff HEAD~1` shows the four repairs, and RPT-4 is the rule it skipped — instructions influence, they do not enforce.

Segment 4 — The algorithmic verifier (about 14 minutes)

Say first: that check cost tokens and could differ next time. Every clause of this specification is mechanical (VER-2): a program can decide it from the file alone.

Do: prompt:

Write `check_notes.py`, Python 3, standard library only. Given note paths, it decides R1–R5 and R7 for each; given `--all`, it decides every note in `notes/` and R6 over `index.md`. One line per violation, citing the rule ID; exit 0 if conformant, 1 if there are violations, 2 on a usage error. Its docstring states which specification version it implements and which clauses it decides. Run it with `--all` and report per RPT-2. Confirm the VER-2 table in `process/verification.md` is right for 1.0.0 — R7 is mechanical — and commit as `verifier: check_notes.py for specification 1.0.0`.

Expected: the script written, run (`--all` passes), the VER-2 row for R7 added, a commit.

Do (shell): change the Royce note’s `created:` line to `Sept 2026` in the editor and save; then:

```
python3 check_notes.py --all; echo "exit: $?"
git checkout -- notes/royce-1970-waterfall-paper.md
python3 check_notes.py --all; echo "exit: $?"
```

Expected: one R2 line and exit 1; after the restore, all conformant and exit 0.

[fallback capture] — both runs.

Say: the program cites the same rule IDs the agent cited in Segment 3; the specification was numbered for this. It is deterministic, immediate, and free, and its exit code lets it stand in front of a commit or a build. It decides the mechanical clauses only; at 1.0.0 that is all of them, and judgment clauses arrive in part 2. And it is itself a realization of the specification: if it implements a rule wrongly, its results are wrong in a way its output does not show. That is why every result states which verifier produced it, against which version, deciding which clauses (RPT-2).

If it goes differently: if the live-written verifier has a bug, spend at most two minutes on it and say what it is — “the verifier is a realization that failed conformance; the same loop applies to it” — then run the fallback copy of `starter-102/check_notes.py` from outside the repository.

End of part 1

The repository now holds the specification at 1.0.0, one conformant note, its index entry, the verifier for 1.0.0, and the L01 process files. Part 2 starts from `starter-102/`, which is this state plus a concept of operations and the process documents that govern operations.

Recreating this part yourself (students)

Copy `starter/` somewhere outside the course repository, follow the setup checklist, and run Segments 2–4 with your own Claude Code session. Do not read `sample-inputs/royce-note-filled-nonconformant.md`; write a note of your own carelessly and see what the audit-amended specification catches. Your run will differ — a different list in Segment 2, a differently written verifier in Segment 4. Compare your final `note-format-spec.md` with `starter-102/note-format-spec.md`: two realizations of the same intent.